

Proiectarea și implementarea unui accelerator criptografic AES-128 reconfigurabil pe FPGA

Candidat: ing. Dan-Alexandru BULZAN

Coordonator științific. Eugen GURBAN

REZUMAT

Pe parcursul acestei lucrări se va prezenta metodologia de proiectare a unui nucleu criptografic care implementează algoritmul AES (*Advanced Encryption Standard*).

CUPRINS

1	INTRODUCERE.....	4
1.1	INFORMAȚII GENERALE.....	4
1.2	MOTIVAȚIA LUCRĂRII	5
1.3	SPECIFICAȚII TEHNICE.....	6
1.4	SCOP ȘI OBIECTIVE	7
1.5	STRUCTURA LUCRĂRII	8
2	ANALIZA STADIULUI ACTUAL ÎN DOMENIUL PROBLEMEI	9
2.1	SISTEMELE MODERNE ȘI CRIPTOGRAFIA AES	9
2.2	IMPLEMENTAREA AES PE FPGA	10
2.3	SISTEME FPGA ȘI RECONFIGURARE.....	11
2.4	POZIȚIONAREA SOLUȚIEI PROPUSE	11
3	FUNDAMENTE CRIPTOGRAFICE ȘI ALGORITMUL AES	13
3.1	NOȚIUNI FUNDAMENTALE DE CRIPTOGRAFIE	13
3.2	CRIPTOGRAFIA SIMETRICĂ.....	14
3.3	TIPURILE DE ALGORITMI CRIPTOGRAFICI SIMETRICI	15
3.4	ADVANCED ENCRYPTION STANDARD	18
3.5	STRUCTURA ALGORITMULUI AES-128	20
3.5.1	ADĂUGAREA CHEII DE RUNDĂ.....	20
3.5.2	SUBSTITUȚIA OCTEȚILOR (SUBSTITUTION BOX)	21
3.5.3	DEPLASAREA LINIILOR.....	22
3.5.4	AMESTECAREA COLOANELOR.....	22
3.5.5	DERIVAREA CHEII DE RUNDĂ	23
3.6	OPERAȚIA DE DECRYPTARE AES-128.....	24
3.6.1	SUBSTITUȚIA INVERSĂ	25
3.6.2	INVERSAREA DEPLASĂRII LINIILOR	26
3.6.3	INVERSAREA AMESTECĂRII LINIILOR	26
3.6.4	RUNDELE DE DECRYPTARE.....	27
4	FUNDAMENTE FPGA ȘI ARHITECTURA ZYNQ-7000	28
4.1	ARHITECTURA FPGA	28
4.2	ARHITECTURA ZYNQ-7000	29

1 INTRODUCERE

1.1 INFORMAȚII GENERALE

Informația este unul dintre cuvintele cheie care ies în evidență atunci când analizăm dezvoltarea tehnologică din ultimele decenii. Însă, odată cu îmbunătățirea și diversificarea mijloacelor de transmisie, datorită răspândirii sistemelor de calcul la costuri tot mai reduse, siguranța informațională a devenit mai importantă ca oricând. Într-o lume progresiv mai interconectată din punct de vedere digital, schimbul continuu de informații este o certitudine. Însă, integritatea și autenticitatea informațiilor nu pot fi garantate în lipsa unei infrastructuri de securitate adecvate.

Astfel, a apărut necesitatea dezvoltării unui sistem de standarde care să opereze în favoarea minimizării riscului asociat transferului de informații. Aceste standarde sunt construite în jurul conceptelor de securitate și al tehnologiilor care le facilitează implementarea și adoptarea. Mai mult decât atât, standardizarea permite analizarea, respectiv evaluarea gradului de securizare al unui sistem.

Criptologia este aria de studiu care produce conceptele utilizate de standardele de securitate. Aceasta studiază dezvoltarea tehnicilor de securizare a informației, în prezența unui adversar. Motivațiile și scopurile adversarilor vizează o gamă largă. Unii adversari sunt preocupați de destabilizarea politică, pe când alții practică furtul de date dintr-o motivație strict financiară. În plus, nu toți adversarii dispun de aceleași resurse și vectori de atac. Suprafețele prin intermediul cărora o entitate malițioasă atacă un sistem sunt, la rândul lor, extrem de diverse.

Criptografia modernă este construită în jurul unui nucleu matematic. Astfel, operațiile de criptare și decriptare sunt adesea intensive computațional. Însă, chiar vectorii de propagare a informației, și anume, sistemele de calcul sunt, prin natura lor, potrivite pentru efectuarea acestor operații într-un mod rapid și eficient.

Un algoritm criptografic poate fi implementat la nivelul unui sistem de calcul prin intermediul a două mari mijloace, și anume, *software* sau *hardware*. Ambele modalități prezintă avantaje și dezavantaje specifice. Implementările software au avantajul de a fi construite în baza nivelului de abstracție furnizat de către sistemul de operare, respectiv de limbajul de programare utilizat. Mai mult de atât, aceste soluții pot fi dezvoltate rapid și la un cost redus, factori predominanți în analiza fezabilității unui proiect.

Aceste lucruri fiind spuse, este greu de imaginat perspectiva în care o implementare hardware poate fi considerată superioară. Hardware-ul este dificil de proiectat și

implementat, fiind necesari dezvoltatori experimentați pentru a fi realizată o soluție adecvată în raport cu un anumit set de constrângeri.

Totuși, există anumite avantaje care nu pot fi reproduse de către o implementare software. Paralelismul este o trăsătură nativă a hardware-ului, spre deosebire de cazul software-ului, unde operațiunile concurente necesită o sincronizare explicită, atât la nivelul accesului la date, cât și la nivelul controlului execuției. Hardware-ul implementează paralelismul într-un mod direct și structurat prin intermediul logicii combinaționale, structurilor secvențiale, automatelor cu stări finite etc.

În cazul aplicațiilor critice, unul dintre cei mai importanți factori este reprezentat de determinism. Odată ce este demarată, execuția unui algoritm implementat la nivel de hardware nu este direct dependentă de mecanismele de organizare ale sistemului de operare ale sistemului gazdă. În plus, un coprocesor criptografic poate lucra în favoarea sistemului de calcul prin preluarea sarcinilor criptografice. Acest lucru are potențialul de a crește nivelul de disponibilitate a sistemului gazdă, dar și de a spori cantitatea de informație care poate fi procesată într-o anumită perioadă de timp.

Este important de specificat că implementările algoritmilor criptografici nu trebuie să fie strict hardware sau software. Adesea, există un grad de suprapunere, fie acesta direct sau indirect, între cele două tipologii de proiectare. Spre exemplu, este posibilă dezvoltarea unui algoritm software care să utilizeze un set de instrucțiuni criptografice, implementate de către procesorul gazdă la nivel hardware.

1.2 MOTIVAȚIA LUCRĂRII

Sistemele de calcul moderne trebuie să facă față unui volum tot mai mare de informații de procesat, fără a permite operațiilor criptografice să devină o problemă de performanță. Mai mult de atât, chiar dacă performanța sistemului nu este afectată în mod direct, încetinirea sau creșterea latenței aferente procedeeleor criptografice poate avea repercusiuni asupra volumului și calității transmisiei de date.

Asigurarea determinismului operațiunilor criptografice este un aspect de mare interes în cazul aplicațiilor din domenii critice. În astfel de situații, reconfigurarea unui sistem hardware poate fi mai eficientă decât modificarea rutinelor software. În plus, determinismul unei soluții hardware, asociat cu o interfață comună de comunicare care poate fi accesată într-un mod uniform de către componenta software, îmbunătățește gradul de predictibilitate al unui sistem.

Circuitele digitale specializate ASIC (*Application-specific integrated circuit*) sunt, în general, implementate pentru a efectua un set de operații specifice, într-un mod determinist. Acestea prezintă o flexibilitate redusă la modificările iterative care pot denatura sau modifica

total structura operațiilor pe care trebuie să le execute. Din această cauză, soluțiile hardware specializate pot deveni rapid nefuncționale din perspectiva procesului implementat.

Una dintre soluțiile acestei probleme este reprezentată de circuitele digitale reconfigurabile de tip FPGA (*Field-Programmable Gate Array*). Datorită aranjamentului intern de care dispun, circuitele logice ale unui astfel de dispozitiv pot fi configurate pentru a implementa o nouă structură hardware. Astfel, în cazul în care necesitățile computaționale ale unui sistem se schimbă atât de mult încât să fie necesare modificări structurale, un nou circuit digital poate fi dezvoltat și implementat în cadrul structurii logice a FPGA-ului.

Anumite dispozitive FPGA pot expune dezvoltatorului funcționalități de configurare parțială sau totală. Configurarea totală constă în modificarea structurii logice a dispozitivului în întregime. În contrast, configurarea parțială este un procedeu complex care implică reconfigurarea unei regiuni specifice a circuitului implementat, nu substituirea acestuia în totalitate. Astfel, configurarea parțială permite menținerea unei interfețe comune în timp ce doar anumite implementări structurale sunt alterate.

Motivația din spatele acestei lucrări este condusă de dorința de a implementa un sistem criptografic reconfigurabil prin utilizarea unui dispozitiv logic programabil. Un astfel de proiect va evidenția diferențele dintre implementările software clasice și cele hardware. Mai mult de atât, procedeu de dezvoltare al dispozitivelor digitale este construit în jurul unor paradigme fascinante, care sunt adesea neintuitive pentru persoanele acclimatizate ciclului de dezvoltare software.

Pentru a menține un grad de corelare ridicat cu tehnologiile criptografice de actualitate, a fost selectat pentru implementare algoritmul AES (*Advanced Encryption Standard*). În acest context, va fi analizat întregul ciclu de dezvoltare aferent proiectării hardware, de la noțiunile abstracte care definesc interfața entității până la sinteză, implementare și metricile asociate.

1.3 SPECIFICAȚII TEHNICE

Acest proiect este dependent de tehnologiile și dispozitivele utilizate în implementarea sa, în special din cauza faptului că sistemele FPGA prezintă diferențe semnificative în ceea ce privește capabilitățile, resursele logice disponibile și suportul tehnic oferit de producător. Componentele tehnice utilizate sunt următoarele:

- Digilent Arty Z7-20 — SoC AMD/Xilinx Zynq-7000 XC7Z020
- Mediul de dezvoltare Vivado, versiunea 2026.1
- Mediul de dezvoltare Vitis 2026.1
- Limbajul VHDL (standard 2008)
- Limbajul C (standard 99)
- Protocolul AXI4-Lite

Alegerea dispozitivului FPGA este de o importanță critică, întrucât specificațiile digitale ale sistemului integrat determină în mod direct limita de complexitate în care circuitele implementate trebuie să se încadreze. Mai mult decât atât, chiar dacă diferite dispozitive integrate utilizează același SoC, în cazul de față Zynq-7000, pachetele de dezvoltare nu oferă mereu aceleași funcționalități și suprafețe de interfață. În anumite situații nefericite, este posibil ca un kit să fie complet inutilizabil din cauza neintegrării acestuia la nivelul mediului de dezvoltare.

Avantajul sistemului SoC Zynq-7000 XC7Z020 este integrarea unui sistem de procesare (*PS – Processing System*), reprezentat de microprocesorul ARM Cortex 9, împreună cu logica programabilă (*PL – Programmable Logic*). Această arhitectură oferă posibilitatea de a asocia aplicațiile software cu un controler sau dispozitiv periferic reconfigurabil. De asemenea, logica programabilă poate fi utilizată pentru a extinde funcționalitățile hardware ale microprocesorului. În cazul de față, perimetrul logic programabil va fi utilizat pentru implementarea unui coprocesor criptografic.

Limbajul VHDL (*VHSIC HDL*) va fi folosit pentru realizarea unei imagini descriptive a circuitului integrat. Mediul de dezvoltare Vivado, împreună cu tehnologiile de analiză, sinteză și implementare asociate, va fi utilizat pentru verificarea, validarea și generarea coprocesorului criptografic.

Pentru a face posibilă utilizarea dispozitivului generat, acesta va fi împachetat într-o interfață AMBA AXI4-Lite. Comunicarea dintre sistemul de procesare și logica programabilă va fi efectuată prin dezvoltarea unui firmware, prin intermediul limbajului de programare C, în cadrul mediului de dezvoltare Vitis.

1.4 SCOP ȘI OBIECTIVE

Scopul acestei lucrări este dezvoltarea abilităților practice de proiectare și implementare a algoritmilor criptografici la nivel de hardware. Implementarea în cadrul unei platforme FPGA necesită familiarizarea cu un proces de dezvoltare specific. În cazul de față, procesul este bazat, în primul rând, pe respectarea cerințelor standardului criptografic implementat. Însă, mai mult de atât, trebuie acordată o atenție deosebită metricilor de analiză ale comportamentului circuitului integrat proiectat. Inițial, aceste metrice pot fi nefamiliare și neintuitive, mai ales pentru un proiectant neexperimentat. Orice neclaritate sau nerespectare a mecanismelor sistemului digital poate duce la propagarea unor erori, mai mult sau mai puțin subtile, care au potențialul de a afecta buna funcționare a circuitului integrat.

De asemenea, un astfel de proiect oferă un contact substanțial cu aria criptografiei bazate pe hardware. Implementarea algoritmului AES, la nivelul unui coprocesor criptografic, demistifică aparenta complexitate pe care operațiile hardware o pot dobândi. Ulterior, integrarea coprocesorului cu restul sistemului SoC Zynq-7000, prin utilizarea

interfeței AXI4-Lite, duce la dezvoltarea unei bune imagini de ansamblu în ceea ce privește organizarea arhitecturală a sistemelor integrate moderne. Datorită faptului că proiectul prevede și dezvoltarea unui firmware prin care se realizează transferul de date între logica programabilă și nucleele ARM, sistemul va fi implementat la toate nivelurile de abstractizare.

Obiectivele lucrării sunt construite în jurul proiectării și implementării nucleului criptografic AES-128 în limbajul VHDL. Această implementare necesită, în primul rând, înțelegerea teoriei matematice în baza căreia algoritmul este constituit. În urma analizei și transformării operațiilor matematice în structuri logice combinaționale și secvențiale, este necesară validarea funcțională a nucleului criptografic.

Validarea va fi realizată prin intermediul a două etape. Prima este reprezentată de testarea generală a logicii prin utilizarea entităților de test. Cea de a doua etapă a validării va fi efectuată în urma integrării coprocesorului criptografic. Pentru a realiza acest obiectiv vor fi utilizați vectorii de test puși la dispoziție de NIST (*National Institute of Standards and Technology*).

În urma procedurii de dezvoltare și validare, va fi analizată posibilitatea reconfigurării dinamice parțiale sau totale a coprocesorului, în baza mecanismelor DFX (Dynamic Function eXchange) oferit de către AMD/Xilinx.

Este important de menționat faptul că la fiecare pas al procesului de dezvoltare, vor fi analizate artefactele și rapoartele produse de către mediul de dezvoltare. Lipsa unei bune înțelegeri sau chiar ignorarea totală a acestor metricilor poate rezulta într-un circuit digital al cărui comportament real este categoric diferit față de cel simulat.

1.5 STRUCTURA LUCRĂRII

Lucrarea urmărește progresiv trecerea de la fundamentele teoretice ale criptografiei către proiectarea și validarea soluției hardware propuse. Sunt prezentate mai întâi stadiul actual al domeniului și noțiunile necesare înțelegerii algoritmului AES-128, urmate de fundamentele arhitecturilor FPGA, ale sistemului Zynq-7000 și ale comunicației dintre sistemul de procesare și logica programabilă.

Partea practică tratează implementarea nucleului AES-128 în VHDL, integrarea acestuia prin AXI4-Lite și controlul din procesorul ARM. În final sunt prezentate metodele de verificare, rezultatele obținute și direcțiile de dezvoltare ulterioară.

2 ANALIZA STADIULUI ACTUAL ÎN DOMENIUL PROBLEMEI

2.1 SISTEMELE MODERNE ȘI CRIPTOGRAFIA AES

Sistemele criptografice moderne sunt construite cu scopul de a facilita schimbul securizat de informație între interlocutori. Protejarea confidențialității și integrității datelor reprezintă o cerință fundamentală pe care sistemele informatice trebuie să o îndeplinească. Aceste directive sunt îndeplinite prin utilizarea unei suite complexe de operații criptografice, care adesea acționează ierarhic la mai multe dintre nivelurile procesului de transmisie al datelor.

Operațiile de criptare pot fi realizate prin intermediul a două categorii principale de algoritmi criptografici: algoritmi cu cheie simetrică și algoritmi cu cheie asimetrică. Unul dintre cei mai răspândiți algoritmi cu cheie simetrică este AES, standardizat de NIST și utilizat pentru criptarea blocurilor de o dimensiune de 128 de biți. Acest algoritm permite utilizarea cheilor de 128, 192, respectiv 256 de biți[1].

Implementarea algoritmului AES poate fi realizată atât la nivel de software cât și la nivel de hardware. În ceea ce privește ecosistemul software, au fost dezvoltate diverse librării criptografice care implementează o suită vastă de algoritmi. Dintre aceste librării putem aminti *OpenSSL*, aceasta fiind larg utilizată în asigurarea securității traficului de pe internet[2].

Implementările hardware pot fi realizate prin descrierea hardware-ului aferent unui coprocesor criptografic. În cazul sistemelor critice care dispun de facilități logice programabile, utilizarea structurii configurabile este o soluție atractivă de implementare[3].

Mai mult decât atât, setul de instrucțiuni al unor microprocesoare a fost extins de-a lungul timpului pentru a oferi suportul necesar accelerării hardware a operațiilor criptografice. De exemplu, *Intel* oferă suport AES nativ prin intermediul setului de instrucțiuni AES-NI[4].

Astfel de extensii pot fi utilizate de implementările software prin intermediul bibliotecilor criptografice, al funcțiilor oferite de compilator sau al codului scris explicit pentru utilizarea instrucțiunilor respective. În acest mod, o parte dintre operațiile intensive ale algoritmului sunt executate direct de unități hardware specializate ale procesorului.

Este important de specificat faptul că implementările strict software prezintă o serie de dezavantaje specifice. Acestea depind, în mod direct, de resursele computaționale disponibile ale procesorului la un moment dat. În cazul în care sistemul de calcul are de procesat un volum mare de date, operațiile criptografice pot încărca semnificativ microprocesorul, contribuind la creșterea latenței. Mai mult decât atât, din cauza faptului că

execuția unui proces este orchestrată de către dispecerul de procese (*task scheduler*), operațiile criptografice sunt adesea executate într-un mod fragmentat, ca urmare a mecanismului de comutare între procese (*task switching*)[5].

În cazul sistemelor cu cerințe de timp real, aceste aspecte constituie un dezavantaj pentru că determinismul temporal este mai dificil de garantat. Utilizarea unui accelerator hardware permite transferarea sarcinilor criptografice intensive către o structură digitală dedicată care funcționează independent de fluxul de control al microprocesorului principal.

2.2 IMPLEMENTAREA AES PE FPGA

Dispozitivele FPGA reprezintă o soluție intermediară între flexibilitatea implementărilor software și caracterul specializat al circuitelor ASIC. Logica programabilă permite realizarea unor structuri hardware dedicate algoritmului AES, păstrând în același timp posibilitatea modificării ulterioare a arhitecturii implementate.

Dezvoltatorii acestor sisteme criptografice pot gestiona direct modul în care resursele digitale și computaționale sunt alocate. În funcție de constrângerile impuse implementării, structurile secvențiale și combinaționale pot fi modificate, astfel încât totalitatea cerințelor să fie îndeplinite.

În funcție de modul în care resursele hardware sunt utilizate, implementările AES pe FPGA pot adopta mai multe tipuri de arhitecturi. O primă categorie este reprezentată de arhitecturile iterative, în cadrul cărora aceeași structură hardware este reutilizată pentru procesarea succesivă a rundelor algoritmului. Această abordare reduce consumul de resurse logice, însă limitează cantitatea de date procesate, întrucât un bloc de date necesită mai multe cicluri de tact pentru a parcurge întregul proces criptografic.

Arhitectura desfășurată (*unrolled*) este în contrast cu abordarea anterior menționată. În cadrul acesteia, operațiile mai multor sau chiar ale tuturor rundelor AES sunt implementate într-un mod concurrent prin structuri combinaționale distincte. Eliminarea structurii iterative poate produce creșterea debitului de procesare a datelor. Însă, consecința este un consum semnificativ mai ridicat de resurse. Mai mult decât atât, dacă traseul logic critic nu respectă limitele de temporizare, funcționarea corectă a circuitului la frecvența impusă nu poate fi garantată[6].

O extensie a acestei abordări este reprezentată de arhitecturile de tip *pipeline*. Prin introducerea unor registre între etapele succesive ale algoritmului, procesarea mai multor blocuri de date poate fi suprapusă în timp[6]. O astfel de abordare îmbunătățește debitul de procesare a datelor însă introduce o latență comparabilă cu perioada de timp necesară procesării tuturor rundelor. Arhitectura pipeline procesează primul bloc al unui mesaj într-o perioadă de timp similară celei necesare unei arhitecturi iterative. Însă, pe măsură ce pipeline-ul se umple, acesta poate ajunge să furnizeze un bloc criptat la fiecare ciclu de tact.

Alegerea arhitecturii depinde, în consecință, de constrângerile impuse sistemului. Resursele digitale disponibile și parametrii funcționali ai unui dispozitiv FPGA au un rol important în influențarea soluțiilor de implementare. O arhitectură iterativă favorizează economisirea resurselor, în timp ce o implementare pipeline favorizează performanța.

2.3 SISTEME FPGA ȘI RECONFIGURARE

Sistemele SoC FPGA combină într-un singur dispozitiv un sistem de procesare convențional cu o regiune de logică programabilă. O astfel de arhitectură permite asocierea flexibilității software-ului cu posibilitatea implementării unor acceleratoare hardware dedicate. Procesorul poate gestiona operațiile generale ale sistemului, în timp ce logica programabilă este utilizată pentru executarea funcțiilor care beneficiază de paralelism și specializare hardware.

Familia Zynq-7000 reprezintă un exemplu de astfel de arhitectură, integrând un sistem de procesare bazat pe nuclee ARM împreună cu logică programabilă FPGA. Comunicarea dintre cele două regiuni permite controlul și transferul de date către acceleratoarele hardware implementate în logica programabilă.

Un avantaj suplimentar al dispozitivelor FPGA este posibilitatea reconfigurării structurii hardware după implementarea inițială a sistemului. Reconfigurarea poate fi realizată la nivelul întregii logici programabile sau doar asupra unei regiuni prestabilite. În cazul reconfigurării parțiale, o parte a sistemului poate rămâne funcțională în timp ce o altă regiune este înlocuită cu o implementare hardware diferită.

În contextul acceleratoarelor criptografice, această caracteristică poate permite adaptarea funcționalității hardware la cerințele sistemului, fără modificarea completă a platformei. De exemplu, aceeași regiune reconfigurabilă ar putea găzdui, în momente diferite, implementări criptografice distincte sau arhitecturi optimizate pentru alte constrângeri de performanță.

2.4 POZIȚIONAREA SOLUȚIEI PROPUSE

Soluția propusă în cadrul acestei lucrări este bazată pe implementarea unui coprocesor criptografic AES-128 cu arhitectură iterativă. Alegerea acestei arhitecturi are la bază mai multe considerente. În primul rând, un circuit digital iterativ utilizează, în general, un volum mai redus de resurse comparativ cu variantele desfășurate sau de tip pipeline. Reducerea complexității structurii logice facilitează atât procesul de proiectare, cât și etapele de verificare și validare.

În plus, chiar dacă organizarea arhitecturală diferă, operațiile fundamentale ale algoritmului AES rămân aceleași pentru toate cele trei tipuri de implementare. Din acest

motiv, arhitectura iterativă reprezintă un punct de plecare adecvat pentru înțelegerea transformării primitivei criptografice într-o structură hardware.

În al doilea rând, o astfel de arhitectură oferă un reper util pentru analiza unor direcții viitoare de optimizare. Rezultatele obținute în ceea ce privește utilizarea resurselor, latența și debitul de procesare pot fi comparate ulterior cu cele ale unor implementări desfășurate sau de tip pipeline.

3 FUNDAMENTE CRIPTOGRAFICE ȘI ALGORITMUL AES

3.1 NOȚIUNI FUNDAMENTALE DE CRIPTOGRAFIE

Criptologia este o disciplină fascinantă care se află la intersecția matematicii, ingineriei electronice și a tehnologiei informației. În perioada premergătoare anilor 1970, aplicațiile criptografice erau aproape exclusiv limitate la sistemele critice militare sau guvernamentale [7]. Însă, odată cu răspândirea tehnologiilor digitale și, drept urmare, a mijloacelor de acces la informație, criptografia a început să fie integrată treptat în numeroase aspecte ale vieții cotidiene.

Ramificațiile criptologiei, fără o analiză a infrastructurii digitale contemporane, nu sunt imediat evidente. În lipsa standardelor adecvate în baza cărora să se poată asigura *confidențialitatea*, *integritatea* și *autenticitatea* datelor, transferul vast de informație care constituie cadrul suport al lumii digitale nu ar fi posibil. Aceste standarde sunt dezvoltate ținând cont de cerințele de securitate și capabilitățile computaționale ale sistemelor reale.

La un nivel fundamental, numeroase operații criptografice urmăresc prelucrarea unui mesaj prin utilizarea unui secret cunoscut doar de interlocutori. Termenul de *cheie criptografică* este folosit pentru a face referire la un astfel de secret.

În funcție de modul în care sunt utilizate cheile criptografice, algoritmi pot fi împărțiți în două categorii principale: algoritmi cu cheie simetrică și algoritmi cu cheie asimetrică. Criptografia simetrică utilizează o cheie secretă comună pentru operațiile de criptare și decriptare, în timp ce criptografia asimetrică utilizează o pereche de chei distincte, publică și privată. Cele două categorii sunt utilizate adesea complementar în cadrul sistemelor criptografice moderne [7].

Dincolo de clasificarea algoritmilor în funcție de modul de utilizare a cheilor, sistemele criptografice sunt construite în jurul unui set de operații fundamentale, fiecare având rolul de a satisface cerințe specifice de securitate.

Operațiile de criptare și decriptare sunt utilizate pentru asigurarea confidențialității informației. Funcțiile *hash* permit obținerea unei amprente digitale a datelor și sunt utilizate frecvent în mecanisme de verificare a integrității. Mecanismele de autentificare a mesajelor și semnăturile digitale extind aceste proprietăți prin permiterea verificării originii și autenticității informației. În plus, protocoalele de stabilire a cheilor permit interlocutorilor să obțină în mod sigur materialul criptografic necesar comunicației[8].

O mare parte a construcțiilor criptografice moderne nu pot fi demonstrate ca fiind sigure în sens absolut. Un astfel de demers ar necesita demonstrarea dificultății unor probleme computaționale, precum cea a logaritmului discret, pentru care nu sunt cunoscute metode

eficiente de rezolvare în cazul parametrilor utilizați în practică. Din această cauză, sistemele criptografice moderne sunt construite pe baza unor presupuneri matematice privind dificultatea anumitor probleme. O astfel de abordare oferă beneficiul modularității sistemelor criptografice care sunt proiectate pe suportul oferit de anumite presupuneri matematice fundamentale. Separarea acestor niveluri permite înlocuirea componentelor individuale atunci când apar noi rezultate matematice sau când nivelul de încredere asociat unei anumite construcții este diminuat [8]. Spre deosebire de abordarea prezentată anterior, un sistem criptografic construit fără o justificare formală a securității împotriva unui adversar care dispune de o cantitate finită de resurse computaționale nu poate fi considerat sigur.

3.2 CRIPTOGRAFIA SIMETRICĂ

Algoritmii de criptare simetrică utilizează același set de chei criptografice pentru operațiile de criptare și decriptare. Rolul unui sistem bazat pe criptografia simetrică este de a permite transmiterea informației printr-un canal de comunicație nesecurizat, fără compromiterea confidențialității datelor, atât timp cât cheia criptografică rămâne secretă. Pentru a face posibilă transmiterea datelor, este necesară partajarea cheii utilizate prin intermediul unui mecanism de distribuție securizat. Un astfel de canal poate avea multe forme, de la un schimb de informații efectuat în persoană de către interlocutori, până la utilizarea anumitor protocoale criptografice. Structura ideală a unui astfel de sistem criptografic este prezentată în cadrul Figurii 3.1.

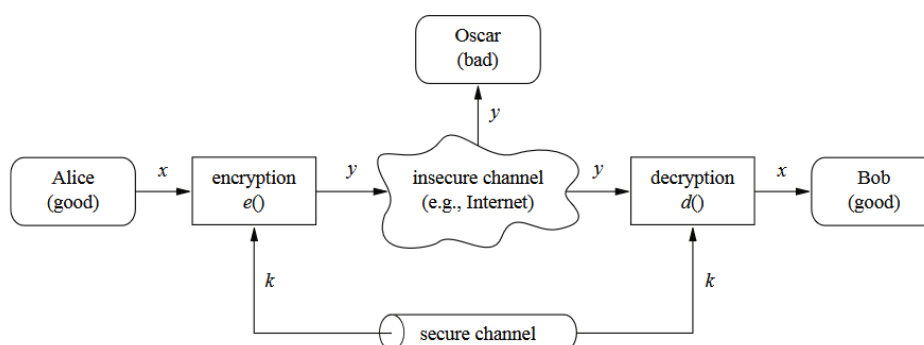


Figura 3.1 Structura sistemului de criptare simetric [9]

Este important de specificat faptul că algoritmii de criptare sunt public disponibili. În primă instanță, acest lucru poate părea contraintuitiv însă, un algoritm public poate fi testat de agenți terți, împotriva unei game largi de atacuri și capacități computaționale. Ca regulă generală, singurul lucru care trebuie menținut secret într-un sistem criptografic este cheia [7].

Procesul de criptare în sine tratează în principal problema confidențialității. Un sistem criptografic robust este compus din mai multe mecanisme primitive, a căror aplicare succesivă are rolul de a asigura atât integritatea, cât și autenticitatea mesajului transmis. În lipsa unor astfel de măsuri, un atacator are posibilitatea de a se interpune între părțile comunicante și de a altera structura mesajului transmis. Într-un astfel de scenariu, modificarea datelor criptate nu este întotdeauna detectabilă în urma procesului de decriptare, ceea ce poate afecta fluxul comunicării sau chiar sisteme automate care depind de analiza promptă și predictibilă a unui flux de date.

3.3 TIPURILE DE ALGORITMI CRIPTOGRAFICI SIMETRICI

Din punct de vedere structural, algoritmi simetrici sunt împărțiți în două mari categorii, și anume:

- Algoritmi simetrici de tip cifru de flux (*stream cipher*)
- Algoritmi simetrici de tip cifru bloc (*block cipher*)

Cifrurile de flux (*stream ciphers*) realizează criptarea prin procesarea individuală a biților sau octeților mesajului. În cazul cifrurilor de flux sincrone, un flux de biți pseudo-aleatoriu este generat pe baza cheii criptografice și este combinat cu biții textului în clar prin intermediul operației *sau exclusiv* (*XOR*). În schimb, cifrurile de flux asincrone utilizează, pe lângă cheia inițială, și biți proveniți din criptotextul generat anterior pentru determinarea fluxului de cheie. Majoritatea cifrurilor de flux utilizate în practică adoptă o arhitectură sincronă.

Structura fundamentală a unui cifru stream este prezentată la nivelul figurii 3.2

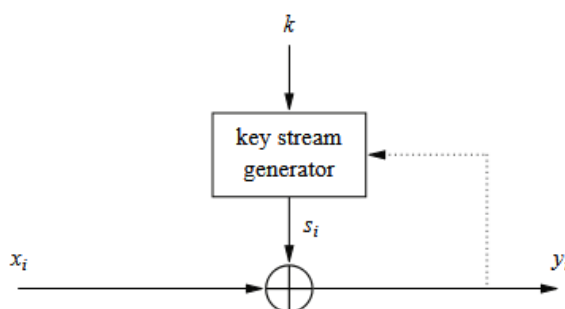


Figura 3.2 Structura unui cifru de tip stream [10]

Datorită consumului redus de resurse computaționale, cifrurile de flux sunt utilizate frecvent în aplicații care impun constrângeri specifice de implementare, precum dispozitivele integrate sau sistemele de telecomunicații. Un exemplu este reprezentat de cifrul de flux A5/1, utilizat în cadrul standardului GSM pentru criptarea comunicațiilor telefonice [7]. Un alt algoritm cunoscut din această categorie este RC4, remarcat datorită simplității structurii sale și a facilității de implementare.

Securitatea cifrurilor stream este, în mod particular, dependentă de modul în care cheia de flux este generată [7]. Mai mul decât atât, majoritatea resurselor computaționale ale unui cifru stream sunt alocate blocului de generare a cheii de flux. Scopul acestui bloc este de a genera biții care vor fi utilizați în procesul de criptare astfel încât aceștia să aibă aspectul unui flux de numere aleatoriu. Există numeroase tehnici de a produce astfel de fluxuri însă, analizarea acestora este dincolo de scopul acestei lucrări.

Spre deosebire de cifrurile de flux, care realizează transformarea succesivă a biților sau octeților individuali, cifrurile bloc operează asupra unor grupuri de date cu dimensiune fixă. Această abordare permite construirea unor structuri criptografice complexe bazate pe transformări repetate aplicate asupra întregului bloc de informație [8]. Structura generală a unui cifru bloc este prezentată la nivelul figurii 3.3.

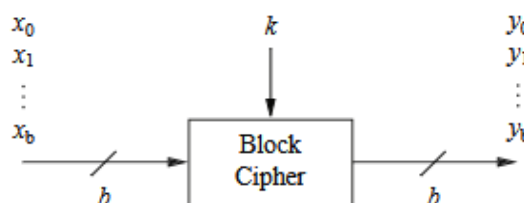


Figura 3.3 Structura fundamentală a cifrului bloc [11]

Modul de operare al unui cifru bloc descrie modalitatea de aplicare a transformațiilor succesive astfel încât să fie posibilă criptarea, respectiv decriptarea fluxurilor continue de date. Printre cele mai întâlnite moduri se număra următoarele:

- Electronic Codebook (ECB)
- Cipher Block Chaining (CBC)
- Cipher Feedback (CFB)
- Counter (CTR)

În lipsa utilizării unui mod adecvat de operare, anumite tipare prezente în datele inițiale pot persista în structura criptotextului. Acest aspect este evident în cazul modului ECB (*Electronic Codebook*), unde blocuri identice de text în clar produc blocuri identice de text criptat, permițând unui agent malițios să extragă informații structurale despre datele procesate.

Cifrurile bloc nu realizează, în general, o singură transformare asupra blocului de date, ci aplică succesiv mai multe operații criptografice organizate sub forma unor runde. Repetarea acestor transformări permite obținerea proprietăților de confuzie și difuzie necesare unui algoritm criptografic modern.

Paradigma de confuzie-difuzie a fost introdusă de către *Claude Shannon*. Aceasta constă în construcția unei permutări $F(x)$ care să posede un aspect pseudo-aleatoriu. Permutarea majoră $F(x)$ este construită din blocul original x prin aplicarea unor permutări minore succesive, reprezentate de funcția $f_i\{k\}$, asupra blocurilor componente ale textului original [8].

De exemplu, în cazul în care dorim generarea unei permutări $F(x)$ pentru un bloc x de 128 de biți, putem defini funcția $f_i\{k\}$ astfel încât aceasta să genereze o permutare pentru fiecare sub-bloc de 8 biți al lui x . Astfel, prin concatenarea celor 16 permutări generate, vom obține rezultatul final $F(x)$. Formula 3.1 definește în mod explicit acest proces [8].

$$F_k(x) = f_1(x_1) \parallel \cdots \parallel f_{16}(x_{16}) \quad (3.1)$$

Operația descrisă nu va produce însă rezultatul pseudo-aleatoriu dorit. În cazul în care aplicăm formula 3.1 asupra a două blocuri x , respectiv y care diferă doar prin primul bit, blocurile $F(x)$, respectiv $F(y)$ vor fi diferite doar la nivelul primului octet permutat. Această particularitate este o consecință a faptului că funcție de permutare $f_i\{k\}$ este bijectivă.

Din această cauză, este necesară introducerea unui alt bloc de permutare care să efectueze operația de *difuzie* prin intermediul căreia biții rezultatului final sunt permutați între ei. Introducerea pasului de difuzie permite propagarea schimbărilor efectuate prin permutarea generată de către funcția $f_i\{k\}$, procedură numită *confuzie*.

Aceste transformări contribuie la obținerea ceea ce este cunoscut sub denumirea de *efect de avalanșă*. Mai specific, o modificare minoră a datelor de intrare, precum inversarea unui singur bit, determină o schimbare semnificativă la nivelul rezultatului final. Această proprietate este esențială pentru împiedicarea identificării unor relații directe între textul în clar și textul criptat.

Aspectul general al transformărilor de difuzie-confuzie aplicate unui bloc cu dimensiunea de 64 de biți este prezentat în Figura 3.3. Structura acestor transformări este determinată în mod direct de cerințele de securitate și de fundamentele matematice pe baza cărora este proiectat un anumit algoritm criptografic. Mai mult decât atât, anumite construcții criptografice prezintă o arhitectură modulară, care permite modificarea sau înlocuirea componentelor interne responsabile de realizarea acestor transformări.

Majoritatea cifrurilor bloc moderne utilizează principiile de difuzie și confuzie prin intermediul unor structuri interne compuse din transformări de substituție, permutare și amestecare a datelor. Printre algoritmii reprezentativi din această categorie se numără DES, 3DES, AES, Blowfish și Twofish.

Dintre algoritmii de criptare simetrică pe blocuri, AES reprezintă unul dintre cele mai utilizate standarde criptografice moderne. Acesta a fost proiectat pentru a înlocui algoritmii anteriori, precum DES și 3DES, oferind un nivel superior de securitate și posibilitatea implementării eficiente atât software, cât și hardware. În continuare va fi prezentată structura algoritmului AES-128 și operațiile care stau la baza procesului criptografic.

3.4 ADVANCED ENCRYPTION STANDARD

Institutul Național al standardelor din Statele Unite (NIST) a anunțat la începutul anului 1997 că urmează să organizeze o competiție pentru selectarea unui nou cifru bloc, numit *Advanced Encryption Standard*. Scopul acestui nou cifru era înlocuirea standardului învechit *Data Encryption Standard* (DES). Un total de 15 algoritmi diferiți au intrat în competiție. În baza acestora a fost formulată o listă de 5 finaliști care cuprindea următoarele cifruri:

- Rijndael
- Serpent
- Twofish
- RC6
- MARS

Algoritmul Rijndael a fost desemnat câștigător al competiției. Deși niciuna dintre opțiunile anterior menționate nu prezenta vulnerabilități semnificative de securitate,

particularitățile de implementare ale cifrului Rijndael, printre care se remarcă performanța în implementările hardware, eficiența și flexibilitatea arhitecturală, au favorizat selectarea acestuia.

Cifrul AES operează asupra unor blocuri de date cu o dimensiune fixă de 128 de biți. Cheia de criptare poate avea o lungime de 128, 192 sau 256 de biți, fiecare variantă determinând utilizarea unui număr diferit de runde de transformare. Astfel, AES-128 utilizează 10 runde, AES-192 utilizează 12 runde, iar AES-256 utilizează 14 runde pentru procesul de criptare și decriptare.

Spre deosebire de algoritmul pe care l-a înlocuit, și anume DES, arhitectura cifrului AES nu este bazată pe o rețea de tip *Feistel*, ci pe o structură de tip *Substitution-Permutation Network* (SPN) [8]. Aceasta este construită prin aplicarea succesivă a unor transformări de substituție și difuzie, cu scopul obținerii efectului de confuzie și difuzie specific algoritmilor criptografici moderni.

Blocul de date de 128 de biți este reprezentat sub forma unei matrici de 4×4 octeți, denumită *stare* (state). Cei 16 octeți care alcătuiesc blocul sunt organizați în ordine de coloană (*column-major order*), fiecare coloană conținând câte 4 octeți. În urma fiecărei transformări, matricea de stare este modificată succesiv până la obținerea blocului criptat.

Astfel, operațiile de transformare efectuate asupra matricii de stare, de către algoritmul AES, sunt următoarele:

1. **Adăugarea cheii de rundă:** fiecare rundă a algoritmului AES necesită derivarea unei chei de rundă k_r , în baza cheii originale k . Procesul de generare a cheii de rundă diferă în funcție de lungimea cheii utilizate.

2. **Substituția octeților:** fiecare dintre cei 16 octeți ai matricii de stare sunt înlocuiți de către un alt octet în baza unui tabel de substituție. Acest tabel de substituție este o operație bijectivă în câmpul Galois $GF(2^8)$ [7].
3. **Deplasarea liniilor:** liniile matricii de stare sunt deplasate la stânga în următorul fel: prima linie rămâne neschimbată, a doua este deplasată cu un pas, a treia cu doi pași și cea de a patra cu trei pași. Operația de deplasare este ciclică și valorile octeților individuali sunt păstrate.
4. **Amestecarea coloanelor:** pentru a finaliza operația de difuzie, se aplică o transformare liniară la nivelul fiecărei coloane. Această transformare are proprietatea că pentru două valori care diferă prin $b > 0$ octeți, rezultatul înmulțirii matriciale va prezenta cel puțin 5 – b diferențe [8].

Indiferent de dimensiunea cheii utilizate, ultima rundă AES nu include operația de amestecare a coloanelor (MixColumns). Această particularitate permite menținerea unei structuri similare între transformările utilizate în procesul de criptare și cele inverse utilizate pentru decriptare [7]. Structura generală a algoritmului este prezentată în Figura 3.5.

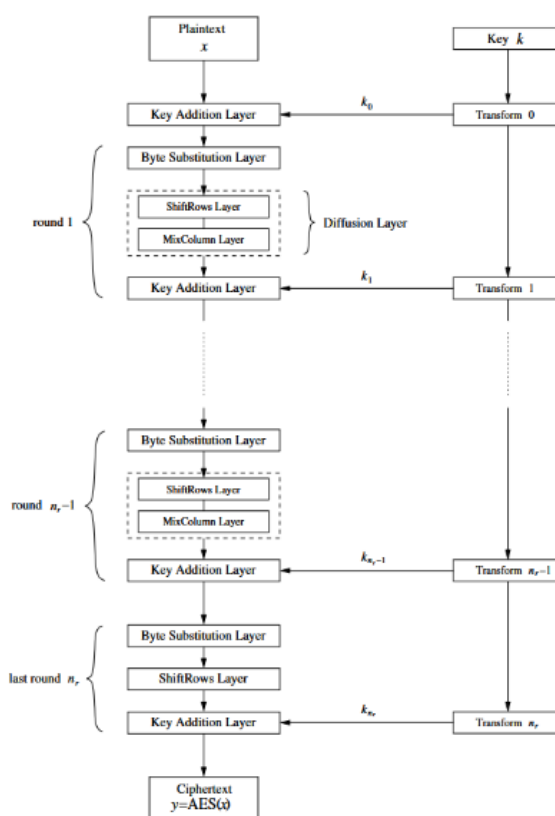


Figura 3.5 Structura algoritmului AES

3.5 STRUCTURA ALGORITMULUI AES-128

Varianta AES-128 a algoritmului Advanced Encryption Standard utilizează o cheie de criptare cu o lungime de 128 de biți și este compusă din 10 runde de transformări succesive. Procesul de criptare are la bază o structură de tip rețea de substituție și permutație, care este construită în baza transformărilor anterior menționate.

Înainte de începerea rundelor propriu-zise este efectuată operația inițială *AddRoundKey*, prin care blocul de intrare este combinat cu prima cheie de rundă. Ulterior, pentru primele nouă runde sunt aplicate succesiv transformările *SubBytes*, *ShiftRows*, *MixColumns* și *AddRoundKey*. Ultima rundă diferă prin eliminarea operației *MixColumns*, conform procedurii anterior prezentat în Figura 3.5.

Pentru înțelegerea modului în care algoritmul AES realizează transformarea blocului de date inițial în blocul criptat, vor fi analizate individual operațiile componente ale unei runde de criptare.

3.5.1 ADĂUGAREA CHEII DE RUNDĂ

Adăugarea cheii de rundă reprezintă cea mai simplă etapă a algoritmului AES. Aceasta constă în efectuarea unei operații *sau-exclusiv* între matricea stării curente și cheia de rundă generată. Operația *sau-exclusiv* este echivalentă cu însumarea celor două valori în corpul Galois $GF(2)$ [7]. Structura abstractă a acestui mecanism este prezentată în Figura 3.6.

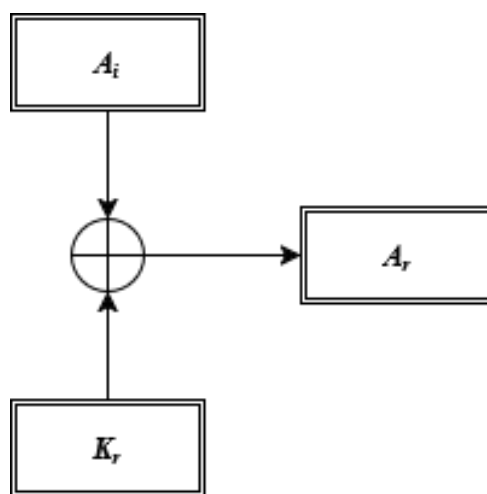


Figura 3.6 Adăugarea cheii de rundă

3.5.2 SUBSTITUȚIA OCTEȚILOR (SUBSTITUTION BOX)

Prima operațiune executată la nivelul unei runde de criptare este reprezentată de substituția octeților. Această transformare introduce neliniaritatea necesară algoritmului AES și are rolul principal de a contribui la proprietatea de confuzie a cifrului. Operația constă în înlocuirea individuală a fiecărui octet din matricea de stare (state) prin intermediul unei tabele de substituție denumită *S-box* (Substitution Box). S-box-ul AES reprezintă o permutare bijectivă a tuturor valorilor posibile pe un octet, având dimensiunea de 256 de elemente. Datorită caracteristicii de bijecție, transformarea efectuată prin substituirea octeților este inversabilă.

Procesul de generare implică calculul inversului multiplicativ al fiecărui element nenul din acest corp, urmat de aplicarea unei transformări afine. Elementul nul este tratat separat, având o valoare fixată în cadrul transformării [9]. Reprezentare matematică a acestei transformări este redată prin formula 3.2.

Type equation here.

În majoritate a cazurilor, atât la nivel de software cât și la nivel de hardware, substituția este implementată prin înlocuirea octeților cu valorile corespunzătoare precalculate [7]. Acestea sunt stocate într-o structură cunoscută sub denumirea de *LUT* (Lookup table).

Totalitatea valorilor de substituție sunt prezentate, în formă hexazecimală, în cadrul tabelului 3.1.

Tabel 3.1 Valorile de substituție

XY	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

3.5.3 DEPLASAREA LINIILOR

Această transformare deplasează, în mod ciclic, octeții ultimelor trei rânduri ale matricei de stare. Prima linie își păstrează structura originală. Linia a doua este deplasată cu o poziție spre stânga, linia a treia cu două poziții, iar ultima linie cu trei poziții. Deoarece deplasările sunt realizate circular, octeții care depășesc limita matricei sunt reintroduși la începutul aceleiași linii [9].

Structura acestei operații, în format matricial, este descrisă în cadrul figurii 3.7.

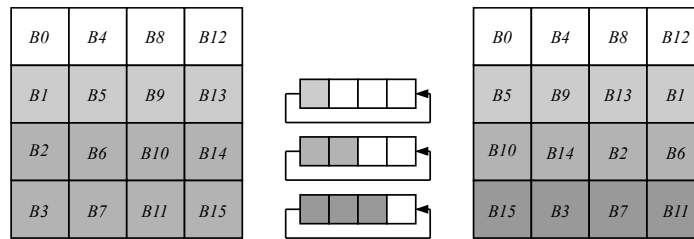


Figura 3.7 Structura operației de deplasare a liniilor

3.5.4 AMESTECAREA COLOANELOR

Această operație reprezintă o transformare liniară aplicată independent fiecărei coloane a matricei de stare. Fiecare coloană este interpretată ca un polinom de grad cel mult trei, ale cărui coeficienți aparțin corpului Galois $GF(2^8)$. Cei patru coeficienți ai

polinomului corespund celor patru octeți ai coloanei, iar fiecare octet este, la rândul său, reprezentat ca un polinom cu coeficienți în $GF(2)$ [9]. Este important de observat faptul că coloanele sunt definite în baza a două grade de ierarhie asupra corpului $GF(2)$.

Coloanele, reprezentate prin polinomul de gradul 3 $s_c(x) = s_{0,c} + s_{1,c}x + s_{2,c}x^2 + s_{3,c}x^3$, sunt înmulțite modulo $x^4 + 1$ cu polinomul $a(x)$ definit la nivelul ecuației 3.3.

$$a(x) = \{02\} + \{01\}x + \{01\}x^2 + \{03\}x^3 \quad (3.3)$$

Coeficienții polinomului $a(x)$, $\{02\}, \{01\}$, respectiv $\{03\}$ sunt echivalenți cu polinoamele x^0 , x și $x + 1$ definite în baza corpului $GF(2)$. Astfel, în formă matricială operația de transformare a coloanelor are următoarea structură:

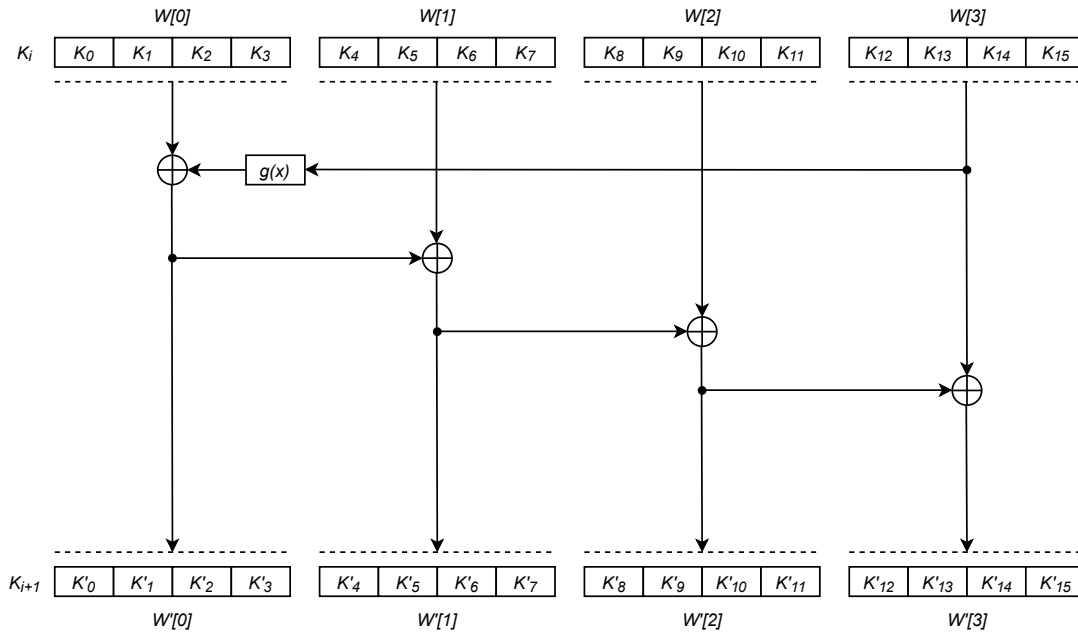
$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}, \quad 0 \leq c < N_b \quad (3.4)$$

3.5.5 DERIVAREA CHEII DE RUNDĂ

Algoritmul AES utilizează cheia K pentru a genera, conform unei transformări matematice, cheile de rundă K_r . Numărul cheilor de rundă generate este egal cu numărul de runde plus o cheie adițională necesară efectuării primei operații de adăugare a cheii. Această primă cheie K_0 este identică cu cheia K .

Cheile de rundă sunt construite în mod recursiv. Astfel, cheia K_{i+1} este generată în baza cheii anterioare K_i [9]. De asemenea, este important de menționat faptul că algoritmi de expansiune a cheilor diferă în funcție de lungimea cheii utilizate. Prin urmare, folosirea unei chei mai lungi implică direct substituirea mecanismului de gestionare a cheilor de rundă.

Pentru o cheie oarecare K de o lungime egală cu 128 de biți, mecanismul de generare cheii de rundă K_r este cel prezentat în Figura 3.8.



Figură 3.8 Generarea cheii de rundă

Fiecare grup de patru octeți formează structuri numite *cuvinte* notate $W[i]$, respectiv $W'[i]$ pentru cuvintele cheii de rundă calculate. Primul cuvânt al noii chei, $W'[0]$ este calculat

Într-un diferit față de restul, prin utilizarea transformării $g(x)$, aplicată ultimului cuvânt al cheii inițiale K_i .

Operația $g(x)$ aplică o rotație asupra celor patru octeți înainte de a-i substitui prin intermediul unui S-box (identic cu cel utilizat de către pasul de substituție) și de adăuga un coeficient. Acest coeficient, numit $RC[i]$, este calculat în funcție de runda curentă conform regulii următoare:

$$RC[i] = x^{i-1} \bmod (x^8 + x^4 + x^3 + x + 1) \quad (3.5)$$

În cazul AES-128, procesul de extindere a cheii generează un total de 44 de cuvinte de 32 de biți, grupate câte patru pentru formarea celor 11 chei de rundă necesare algoritmului. Prima cheie de rundă este reprezentată de cheia inițială, iar următoarele zece sunt obținute recursiv prin aplicarea operațiilor descrise anterior.

Modul de funcționare al funcției $g(x)$ este prezentat în cadrul figurii 3.9.

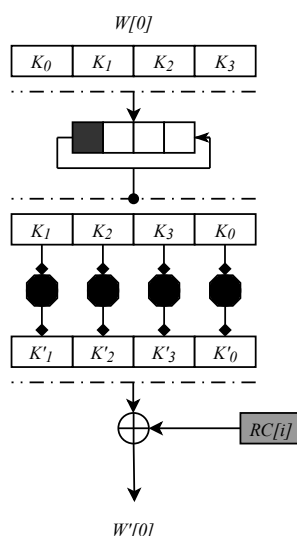


Figura 3.9 Procedura $g(x)$

3.6 OPERAȚIA DE DECRYPTARE AES-128

Operația de decriptare este realizată prin aplicarea transformărilor inverse corespunzătoare etapelor utilizate în procesul de criptare. Algoritmul de extindere a cheii este identic cu cel utilizat în procesul de criptare, însă cheile de rundă sunt aplicate în ordine

inversă, prima operație de decriptare utilizând ultima cheie generată, iar ultima operație utilizând cheia inițială.

3.6.1 SUBSTITUȚIA INVERSĂ

Deoarece funcția de substituție utilizată în AES este bijectivă, fiecărei valori din codomeniul acesteia îi corespunde o singură valoare inițială. Astfel, poate fi definită o funcție inversă capabilă să reproducă valorile inițiale ale octeților prin maparea valorilor substituite.

Operația de inversare este definită la nivelul formulei 3.6. Această necesită aplicarea unei transformări lineare în corpul Galois $GF(2^8)$ [9].

$$A_i = S^{-1}(B_i) = S^{-1}(S(A_i)) \quad (3.6)$$

Pentru a evita calcularea în mod dinamic a transformării inverse, se poate utiliza un tabel precalculat identic cu cel prezentat la nivelul Tabelului 3.2.s

Tabel 3.2 Valorile funcției de substituție inversă

X\Y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

Într-o implementare hardware, această transformare poate fi descrisă sub forma unei memorii de tip LUT, unde octetul de intrare este utilizat drept adresă, iar valoarea memorată reprezintă rezultatul substituției inverse [7].

3.6.2 INVERSAREA DEPLASĂRII LINIILOR

Operația de inversare a deplasării liniilor reprezintă transformarea inversă a operației de deplasare utilizate în procesul de criptare. Aceasta are rolul de a restaura poziția inițială a octeților din matricea de stare prin aplicarea unor deplasări ciclice în direcția opusă.

Astfel, prima linie a matricei rămâne nemodificată, în timp ce următoarele trei linii sunt deplasate circular spre dreapta cu una, două, respectiv trei poziții. Structura acestei transformări este prezentată în Figura 3.10.

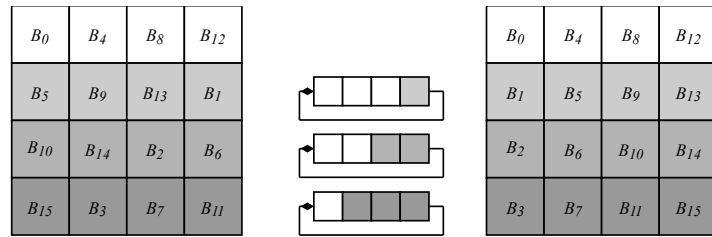


Figura 3.10 Inversarea deplasării liniilor

3.6.3 INVERSAREA AMESTECĂRII LINIILOR

Transformarea inversă este obținută prin înmulțirea polinomului reprezentând coloana de stare $s_c(x)$ cu polinomul invers $a^{-1}(x)$ păstrând aceeași operație de reducere modulo $x^4 + 1$ [8]. Structura polinomului invers este prezentată în formula următoare:

$$a^{-1}(x) = \{0E\} + \{09\}x + \{0D\}x^2 + \{0B\}x^3 \quad (3.7)$$

În formă matricială, modul desfășurat de aplicare al înmulțirii polinomiale este următorul:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} \{0E\} & \{0B\} & \{0D\} & \{09\} \\ \{09\} & \{0E\} & \{0B\} & \{0D\} \\ \{0D\} & \{09\} & \{0E\} & \{0B\} \\ \{0B\} & \{0D\} & \{09\} & \{0E\} \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}, \quad 0 \leq c < N_b \quad (3.8)$$

3.6.4 RUNDELE DE DECRYPTARE

Rundele de decriptare sunt organizate într-o ordine inversă celor aferente procesului de criptare. Similar operației de criptare, procesul începe prin aplicarea operației de adăugare a cheii de rundă, însă utilizând ultima cheie generată K_{10} . Ulterior, fiecare rundă de decriptare aplică transformările inverse corespunzătoare operațiilor utilizate în procesul de criptare.

Structura funcțională a procedurii descrise este prezentată în cadrul Figurii 3.11.

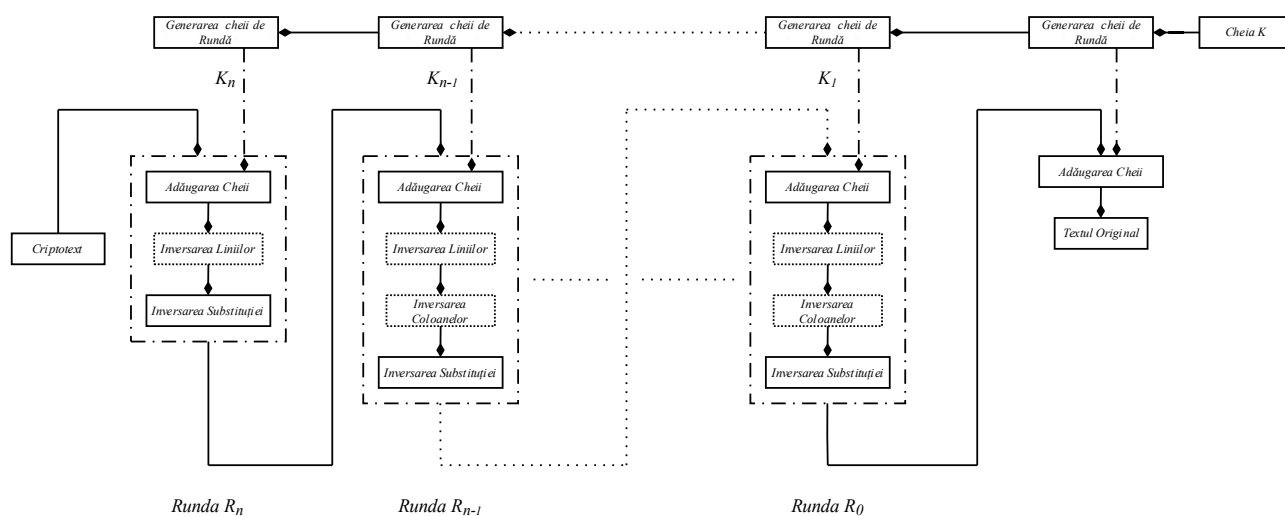


Figura 3.11 Structura procesului de decriptare

În cazul AES-128, procesul de extindere a cheii generează un total de 44 de cuvinte de 32 de biți, grupate câte patru pentru formarea celor 11 chei de rundă necesare algoritmului. Prima cheie de rundă este reprezentată chiar de cheia inițială, iar următoarele zece sunt obținute recursiv prin aplicarea operațiilor descrise anterior.

4 FUNDAMENTE FPGA ȘI ARHITECTURA ZYNQ-7000

4.1 ARHITECTURA FPGA

Dispozitivele FPGA sunt circuite digitale reconfigurabile a căror funcționalitate poate fi definită după fabricație. Structura internă a acestora este alcătuită din resurse logice programabile, elemente de memorare și rețele de interconectare configurabile. Aceste resurse sunt organizate în *slice*-uri, care formează blocurile logice configurabile (CLB) ale dispozitivului [10].

În cadrul logicii programabile a dispozitivelor *Zynq-7000*, blocurile logice configurabile constituie principalele resurse utilizate pentru implementarea circuitelor combinaționale și secvențiale. Acestea integrează LUT-uri, elemente de memorare și logică dedicată operațiilor aritmetice [11].

Rețeaua de interconectare programabilă asigură comunicația dintre diferitele resurse interne ale dispozitivului FPGA, prin stabilirea traseelor necesare propagării semnalelor. Configurarea acestor conexiuni este realizată, în mod obișnuit, automat, în etapa de implementare a proiectului.

Deoarece atât elementele logice, cât și traseele de interconectare introduc întârzieri de propagare, performanța circuitului este influențată nu doar de structura sa logică, ci și de modul în care resursele sunt plasate și rutate în interiorul dispozitivului. Din acest motiv, proiectarea trebuie realizată ținând cont de constrângerile de temporizare și de caracteristicile arhitecturii FPGA utilizate.

Structura logică simplificată a unui slice este prezentată în cadrul Figurii 4.1.

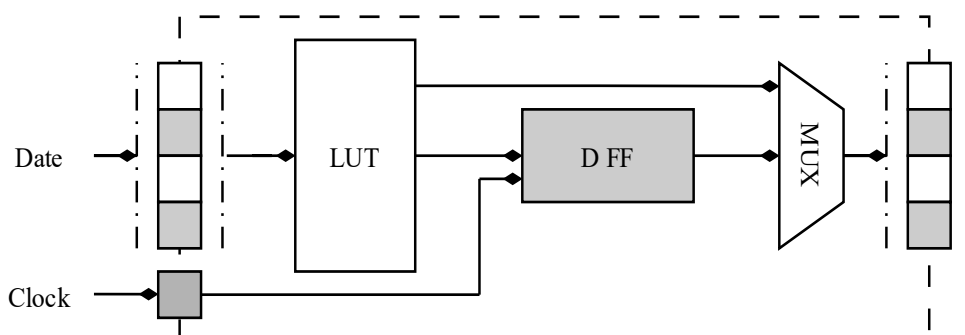


Figura 4.1 Structura conceptuală simplificată a unui slice

Pe lângă resursele logice generale, dispozitivele FPGA integrează și blocuri dedicate anumitor tipuri de operații. Memoriile BRAM (*Block Random Access Memory*) oferă spațiu de stocare intern cu acces rapid și pot fi utilizate pentru implementarea registrelor extinse, a bufferelor sau a memoriilor locale necesare circuitelor digitale.

În mod similar, blocurile DSP (*digital signal processing*) sunt resurse hardware specializate pentru realizarea eficientă a operațiilor aritmetice, în special a înmulțirilor, acumulărilor și altor calcule numerice.

Rețeaua de clock asigură distribuția semnalelor de tact către resursele secvențiale ale dispozitivului FPGA. Aceasta utilizează resurse dedicate de clocking, printre care blocurile de management al tactului (Clock Management Tiles), care permit sinteza frecvenței, compensarea diferențelor de fază (deskew) și controlul jitter-ului [12].

Rețeaua de rutare programabilă realizează interconectarea resurselor interne ale dispozitivului FPGA prin intermediul unor comutatoare. Traseele utilizate pentru propagarea semnalelor sunt configurate în etapa de implementare.

Structura internă simplificată a unui dispozitiv FPGA poate fi observată în cadrul Figurii 4.2.

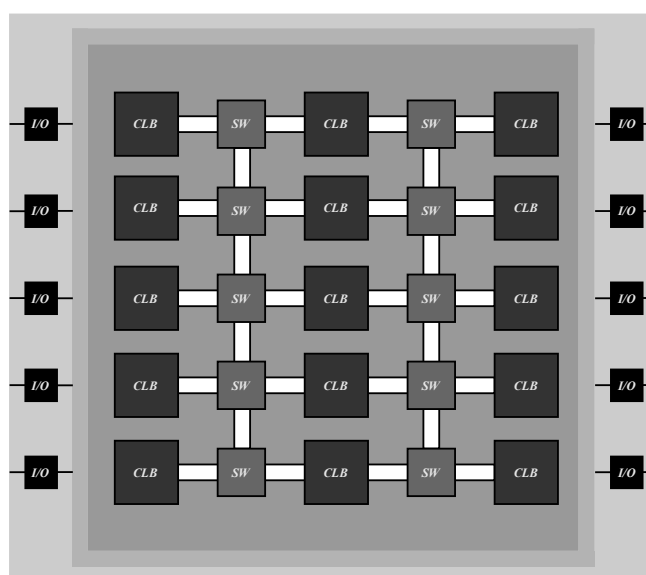


Figura 4.2 Structura conceptuală a arhitecturii FPGA

4.2 ARHITECTURA ZYNQ-7000

Familia de dispozitive *Zynq-7000* combină, în cadrul aceluiași circuit integrat, un sistem de procesare (*Processing System* — PS) și o regiune de logică programabilă (*Programmable Logic* — PL). Această structură permite dezvoltarea unor sisteme în care

funcțiile software sunt executate de procesoarele integrate, iar operațiile care beneficiază de paralelism sau accelerare hardware pot fi implementate la nivelul logicii programabile [11].

Sistemul de procesare integrează o unitate de procesare bazată pe procesoare ARM Cortex-A9, împreună cu subsisteme de memorie, controlere de periferice și interfețe de comunicație.

În configurația utilizată de dispozitivele Zynq-7000 corespunzătoare familiei din care face parte Zynq-7020, unitatea de procesare include două nuclee Cortex-A9. Fiecare nucleu dispune de memorie cache de nivel L1, unitate de management al memoriei și suport pentru procesarea SIMD [11].

Regiunea PL reprezintă componenta FPGA a dispozitivului și permite implementarea unor structuri hardware personalizate utilizând resursele logice prezentate anterior. Din perspectiva unei arhitecturi de accelerare hardware, această separare permite procesorului să gestioneze logica aplicației și transferurile de date, în timp ce operațiile specializate pot fi executate de circuite implementate în PL.

Comunicarea dintre PS și PL este realizată printr-un ansamblu de interfețe dedicate. Pe lângă semnalele de ciclu de tact, întreruperi, control DMA și interfețele de configurare, Zynq-7000 pune la dispoziție mai multe interfețe bazate pe protocolul AXI pentru transferul datelor între cele două regiuni. Acestea includ porturi de uz general (*General Purpose*), porturi de înaltă performanță (*High Performance*) și portul *Accelerator Coherency Port* destinat accesului coerent la subsistemul de memorie al procesoarelor. Diagrama structurală a arhitecturii ZYNQ-700 este prezentată în Figura 4.3.

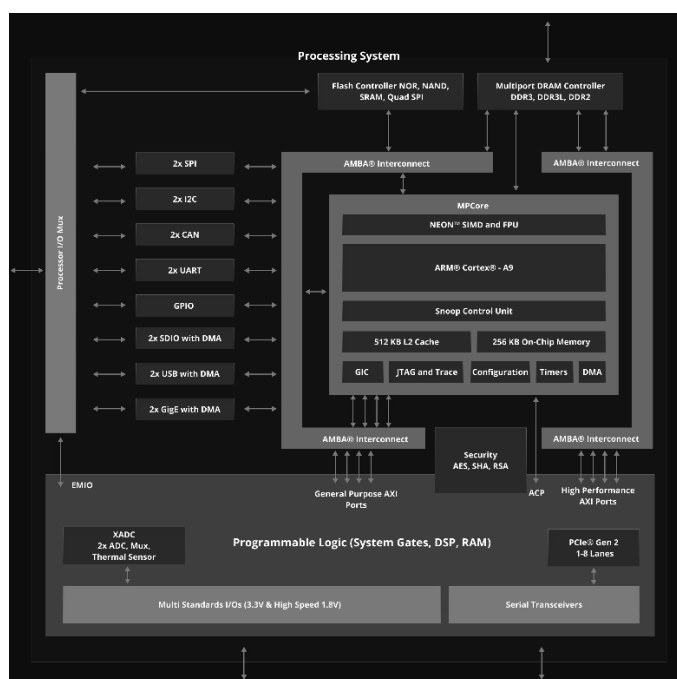


Figura 4.3 Diagrama structurală a SoC-ului ZYNQ-7000 [13]

4.3 PROTOCOLUL AXI4

Protocolul AXI (*Advanced eXtensible Interface*) face parte din familia AMBA dezvoltată de ARM și este utilizat pentru realizarea comunicației dintre componente hardware. În cadrul platformelor Xilinx, protocolul AXI este utilizat pe scară largă pentru interconectarea nucleelor IP (*Intellectual Property*) și pentru comunicația dintre sistemul de procesare și logica programabilă [14].

Familia AXI4 include trei variante principale: AXI4, AXI4-Lite, destinat transferurilor simple și cu debit redus, și AXI4-Stream, utilizat pentru fluxuri continue de date.

AXI4-Lite este destinat în principal accesului la registre de control și stare. Spre deosebire de AXI4, acesta nu suportă transferuri de tip *burst*, fiecare tranzacție fiind asociată unui singur transfer de date [14]. Caracterul său simplificat conduce la o complexitate hardware redusă, făcându-l potrivit pentru controlul perifericelor și al acceleratoarelor hardware.

Interfața AXI4-Lite este organizată în cinci canale independente: canalul de adresă pentru scriere, canalul de date pentru scriere, canalul de răspuns la scriere, canalul de adresă pentru citire și canalul de date pentru citire [14]. Această separare permite efectuarea în paralel a transferurilor de citire și scriere.

Fiecare canal este asociat cu o conduită reprezentată de semnalele *ready* și *valid*. Sursa semnalizează disponibilitatea informației prin activarea semnalului *valid*, iar destinația indică posibilitatea recepționării acesteia prin activarea semnalului *ready*. Transferul are loc atunci când cele două semnale sunt active simultan [14].

Fiind este un protocol de tip *memory-mapped*, AXI4-Lite permite asocierea registrelor unui nucleu IP cu adrese din spațiul de memorie al procesorului. Astfel, software-ul poate controla un accelerator hardware prin operații obișnuite de citire și scriere asupra registrelor asociate acestuia. La un nivel fundamental, comunicarea dintre două dispozitive AXI este efectuată conform Figurii 4.4.

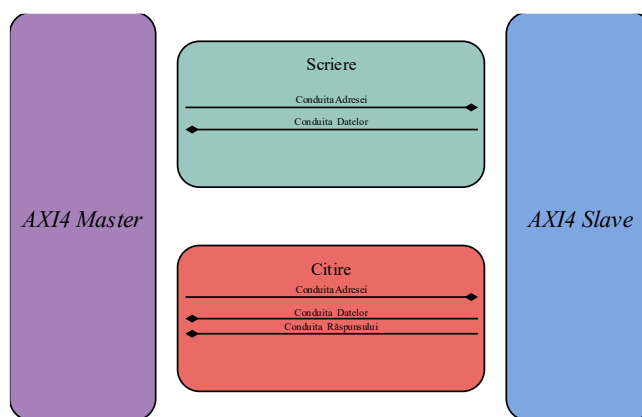


Figura 4.4 Comunicarea dintre două dispozitive AXI4